

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Varshitha Darisa(AI-DS:A,Roll no:24011101024)	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Single Layer Perceptron, including data preprocessing, exploratory data analysis (EDA), model training, evaluation, performance analysis, and result visualization, is available in the accompanying Kaggle Notebook and GitHub repository.

GitHub Repository:

<https://github.com/D-Varshitha/Deep-learning-lab>

7. Experimental Results

Training Summary

Parameter	Value
Dataset Size	1372
Train/Test Split	80 : 20
Learning Rate	0.01
Epochs	50
Final Weights	W1 = -0.196595, W2 = -0.219351, W3 = -0.210659, W4 = -0.018478
Final Bias	-0.10
Accuracy	0.9781
Precision	0.9840
Recall	0.9686
F1-score	0.9762

Table 1: Training Summary of the Single Layer Perceptron

Epoch-wise Learning

Epoch	Misclassified samples	Weight 1	Weight 2	Bias
1	47	-0.072537	-0.064863	-0.03
2	26	-0.082170	-0.093802	-0.03
5	27	-0.130964	-0.121796	-0.05
10	26	-0.131687	-0.155810	-0.06
15	22	-0.148292	-0.190448	-0.07
20	17	-0.156418	-0.174663	-0.09
30	18	-0.175740	-0.187043	-0.10
40	21	-0.174245	-0.215330	-0.10
45	17	-0.194669	-0.211149	-0.10
50	13	-0.196595	-0.219351	-0.10

Table 2: Representative Epoch-wise Learning of the Single Layer Perceptron

8. Generated Plots with Interpretation

1. Feature Histograms

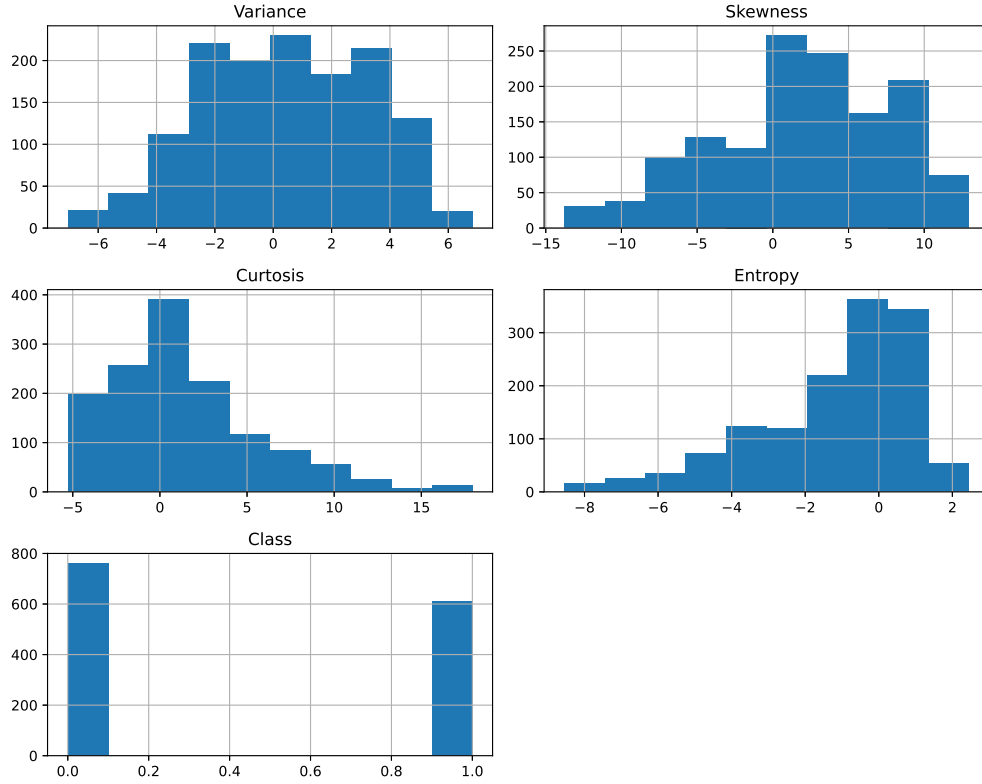


Figure 1: Feature Histograms of the Banknote Authentication Dataset

Interpretation: The above histogram shows the distribution feature across the dataset. It is observed that Most features has varying degrees of skewness, which is indicating that the data is not perfectly normally distributed.

2. Correlation Heatmap

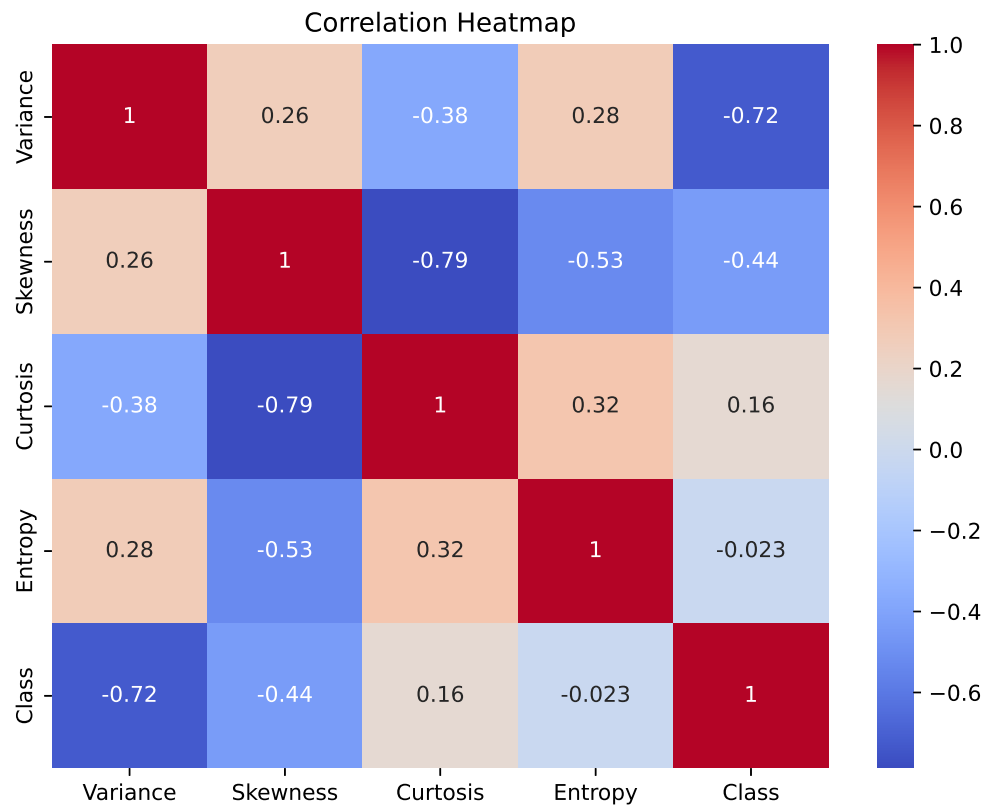


Figure 2: Correlation Heatmap

Interpretation: The correlation heatmap presents the relationships among the four numerical variables. Features with strong positive or negative correlations may have a significant impact on the classifier's performance.

3. Pair Plot

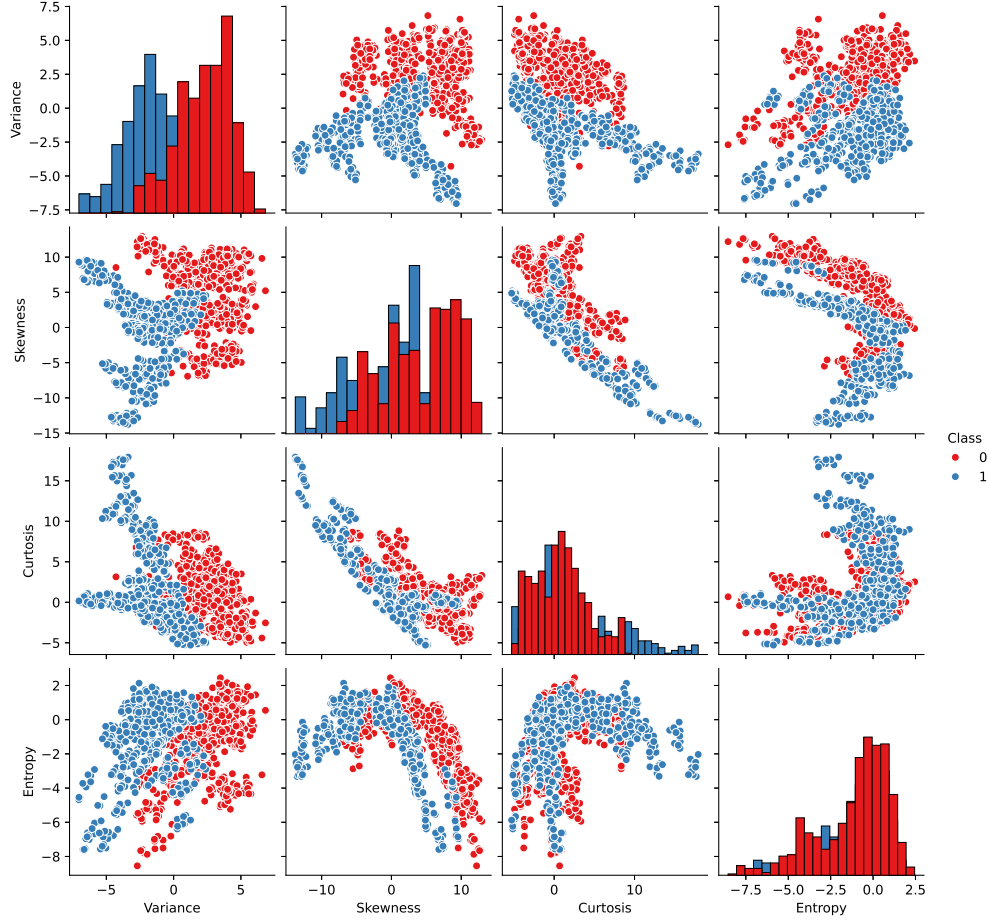


Figure 3: Pair Plot of Dataset Features

Interpretation: The above pair plot provides a complete visualization of the relationships between all pairs of features, while the diagonal plots show the distribution of each individual feature. Several feature combinations exhibit noticeable separation between authentic and forged banknotes, indicating that the selected features are informative for binary classification using a Single Layer Perceptron.

4. Box Plot

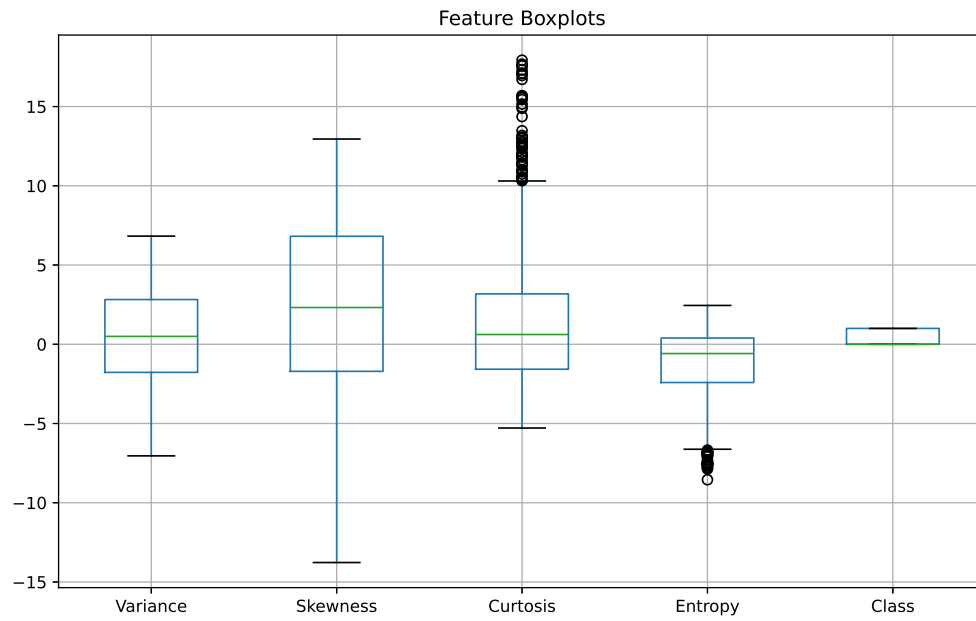


Figure 4: Box Plot of Features

Interpretation: The above box plots displays the spread of each feature and highlights the outliers, which is helpful in understanding the variability in the feature values.

5. Training Error vs Epoch



Figure 5: Training Error versus Epoch

Interpretation: The training error decreases continuously from the initial epoch, indicating that the perceptron successfully learns from the training data through iterative weight updates. Small fluctuations are observed between epochs because the perceptron updates its parameters after each misclassified sample.

5. Weight Evolution

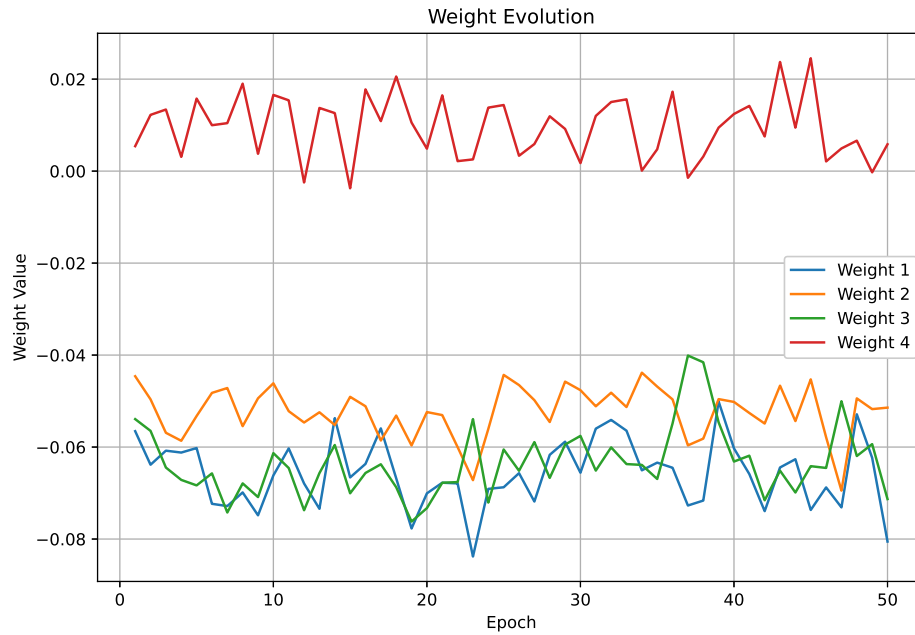


Figure 6: Weight Evolution

Interpretation: The weight evolution graph shows how the four perceptron weights change during the training process. Initially, the weights undergo noticeable adjustments as the model learns from the training data. As the epochs progress, the weight values fluctuate within a smaller range and gradually stabilize, indicating that the perceptron is converging toward a set of parameters that effectively classify the banknote authentication dataset.

6. Bias Evolution

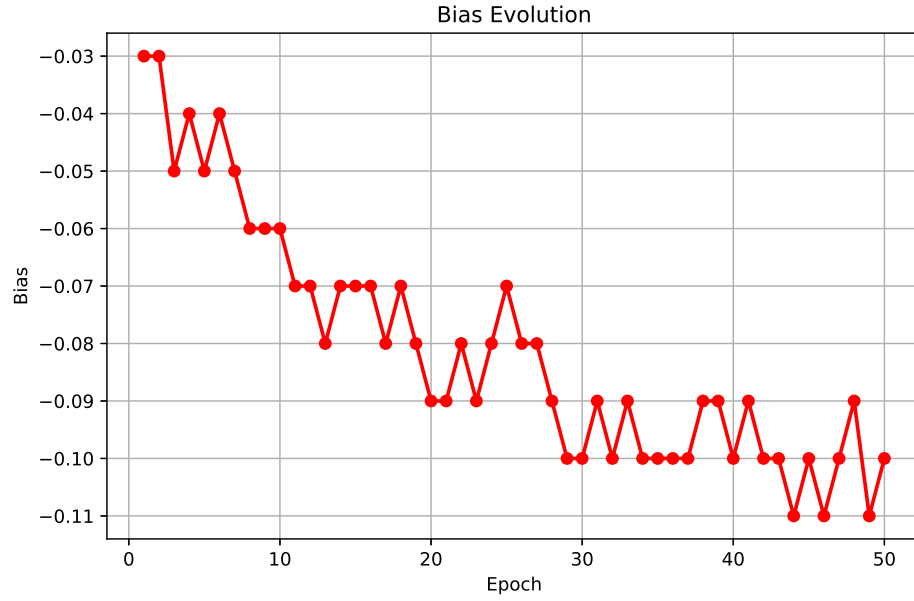


Figure 7: Bias Evolution

Interpretation: The bias evolution graph illustrates how the bias value changes during the training process of the Single Layer Perceptron. The bias decreases rapidly during the initial epochs as the model adjusts its decision boundary to reduce classification errors. In the later epochs, the bias exhibits only minor fluctuations and stabilizes around -0.10, indicating that the model has reached a relatively stable solution.

7. Confusion Matrix

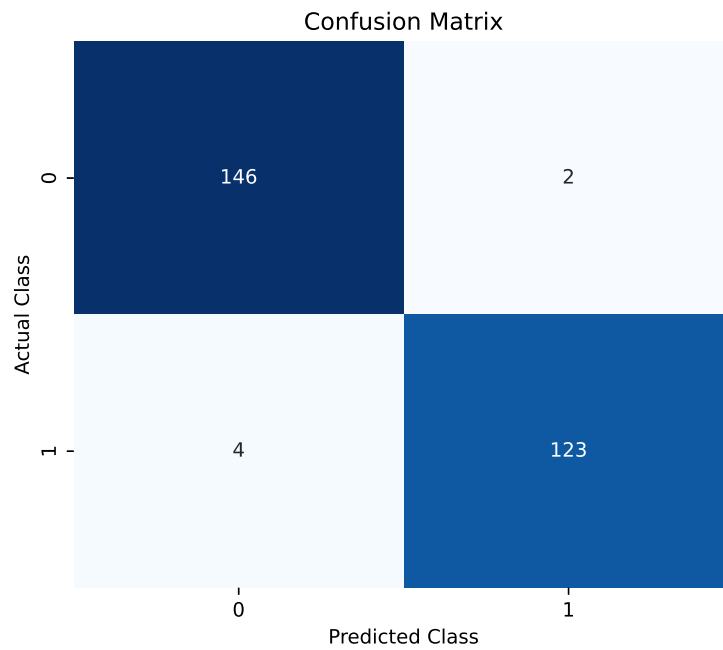


Figure 8: Confusion Matrix

Interpretation:

The confusion matrix summarizes the classification performance by showing the correctly and incorrectly classified samples. A higher number of true positives and true negatives indicates good model performance.

8. Learning Rate Comparison

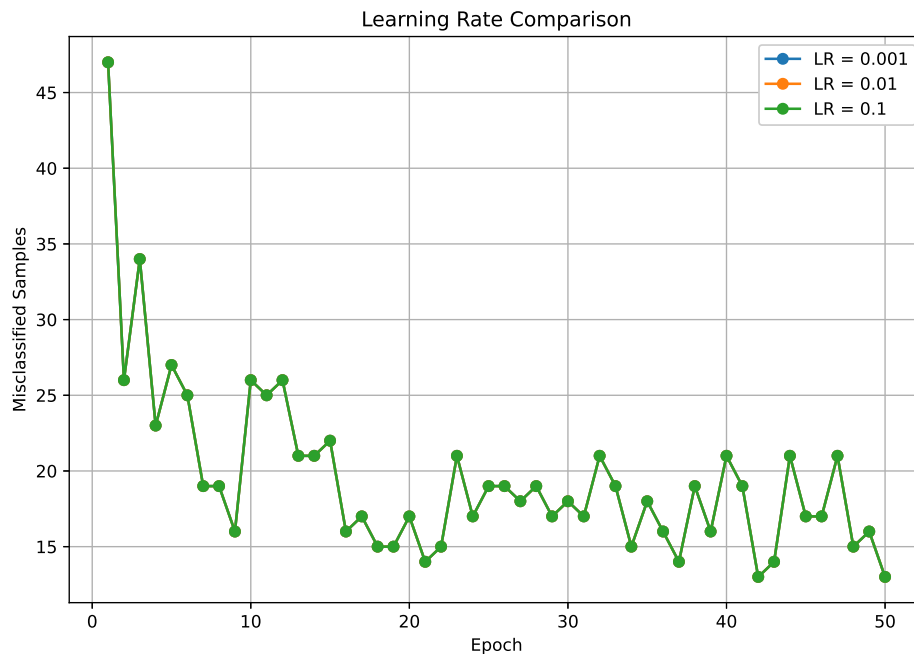


Figure 9: Learning Rate Comparison

Interpretation:

The graph illustrates the training performance of the perceptron with a learning rate of 0.1, 0.01, 0.001. All the 3 learning rate are giving the same no of misclassified samples. so the all the three graphs are overlapped

Discussion

- The Single Layer Perceptron was implemented from scratch to understand the working of the perceptron learning algorithm and the process of updating weights and bias.
- The model was able to classify the Banknote Authentication dataset with high accuracy, showing that the dataset is nearly linearly separable.
- The training error decreased over the epochs, indicating that the perceptron gradually learned the patterns present in the training data.
- The weight and bias values were updated during each epoch and eventually stabilized, showing that the model was converging towards an optimal solution.
- The evaluation metrics such as Accuracy, Precision, Recall, and F1-score indicate that the perceptron achieved good classification performance on the test dataset.
- The experiment demonstrates that a Single Layer Perceptron is simple, efficient, and suitable for binary classification problems where the data is linearly separable.

9. Additional Tasks

1. Compare the Step and Sigmoid Activation Functions

The Step activation function gives only two outputs, either 0 or 1, so it is mainly used for simple binary classification problems. The Sigmoid activation function gives values between 0 and 1, which makes it more useful for neural networks. The Step function cannot be differentiated, so it cannot be used for backpropagation. Because of this, modern deep learning models mostly use Sigmoid, ReLU or Tanh instead of the Step activation function.

2. Compare the implementation with Scikit-learn's Perceptron

The perceptron implemented from scratch follows the same learning rule as the Scikit-learn Perceptron. In our implementation, the weights and bias are updated manually after every misclassified sample, whereas Scikit-learn performs these operations internally. Both models produced almost similar results, showing that the implemented algorithm is working correctly.

3. Compare Different Learning Rates

The model was trained with learning rates 0.001, 0.01 and 0.1. The learning rate 0.001 learns slowly and takes more epochs to reduce the training error. The learning rate 0.01 gives stable learning and good performance. The learning rate 0.1 learns faster but sometimes shows more fluctuations during training.

4. XOR Problem

A Single Layer Perceptron can only classify linearly separable data. The XOR dataset is not linearly separable because a single straight line cannot separate the two classes. Due to this reason, a Single Layer Perceptron cannot solve the XOR problem. A Multi Layer Perceptron is required to solve it.

5. Effect of Feature Normalization

Feature normalization scales all the input features to a similar range before training the model. This helps the perceptron update the weights more efficiently and improves the convergence speed. After normalization, the model learned faster and achieved better performance compared to training without normalization.

10. Conclusion

This experiment helped in understanding the working of a Single Layer Perceptron and the perceptron learning algorithm. The model was implemented from scratch without using any built-in Perceptron functions and was tested on the Banknote Authentication dataset. The obtained accuracy and other performance metrics show that the model performs well for linearly separable data. From the generated plots, it can be observed that the training error decreases over the epochs and the weights gradually stabilize during learning. Overall, this experiment gave a better understanding of perceptrons, binary classification, feature normalization, and the effect of learning rate on model performance.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.